AI Engineering and Claude Architect · Part III, Agents

AI Frameworks, Lesson 2

LangChain and LangGraph

The approval step ran twice in testing. A second "request approval" email went out when the manager clicked approve. The graph had done exactly what its documentation says: on resume, a paused node starts again from its first line.

Module	AI07 of 13
Lesson	2 of 4
Prerequisites	AI06 L1–L2, L7–L8, AI07 L1
Exam relevance	Out of scope
Reading time	~30 minutes

Contents

1. LangChain and LangGraph

Lesson 2 of 4 · AI Frameworks

LangChain and LangGraph

Key idea

LangGraph models a workflow as a graph of shared state, nodes and edges, and saves that state after each step, so a run can pause for a person or resume after a crash. LangChain is the higher-level agent framework built on it. **The value is durable execution and human approval, and the price is learning its execution rules, especially that resumed nodes re-run from the top.**

> *Before: workflows and agents (AI06 L1), durable state (AI06 L7), approval (AI06 L8). Teaches: what the LangChain family is today, state, checkpoints, interrupts, and when you'd choose it. Used by: AI07 L3.*

The hook: seeing the re-run

This runs real LangGraph 1.2.20, with no model at all. The nodes are plain Python, so it shows the framework's mechanics, and says nothing about model quality. Real output from `code/ai07-frameworks/python/langgraph_lab.py`:

```
1. Small refund: no pause
   refunded 7781
2. Large refund: the graph pauses and returns control
   approve node entered, time 1
   paused with: {'question': 'Refund $900?'}
   next node: ('approve',)
   refunds so far: ['7781']
3. Resume, on the same thread, hours later or in another process
   approve node entered, time 2
   refunded 8120
   check node entries: 2 (2 cases, so it didn't re-run on resume)
   approve node entries: 2 (restarted from its first line)
4. A new thread_id knows nothing
   next node on an unknown thread: ()
5. Checkpoints saved for case-2: 5
```

> *Key idea*
>
> **The `approve` node ran its first line twice, while the finished `check` node didn't run again.** So anything before `interrupt()` in a node must be safe to repeat. An email sent there goes out twice. The docs state this clearly, and it's still easy to miss.

This lab used an in-memory saver in one process, so "another process" describes what a durable saver would allow, and I didn't test that here.

Intuition: a board game with a save file

A graph is a board game: squares are **nodes**, arrows are **edges**, and the game state sits on the table, where every player can read it. After each turn, you photograph the table: a **checkpoint**. If the game stops, you restore the photo. If a turn needs a referee's ruling, you pause and wait.

One odd rule: if a turn was interrupted, the player repeats that whole turn from the beginning, not from where they stopped.

Mechanism: the family today

These change quickly. This reflects [the docs](#) read on 5 November 2026.

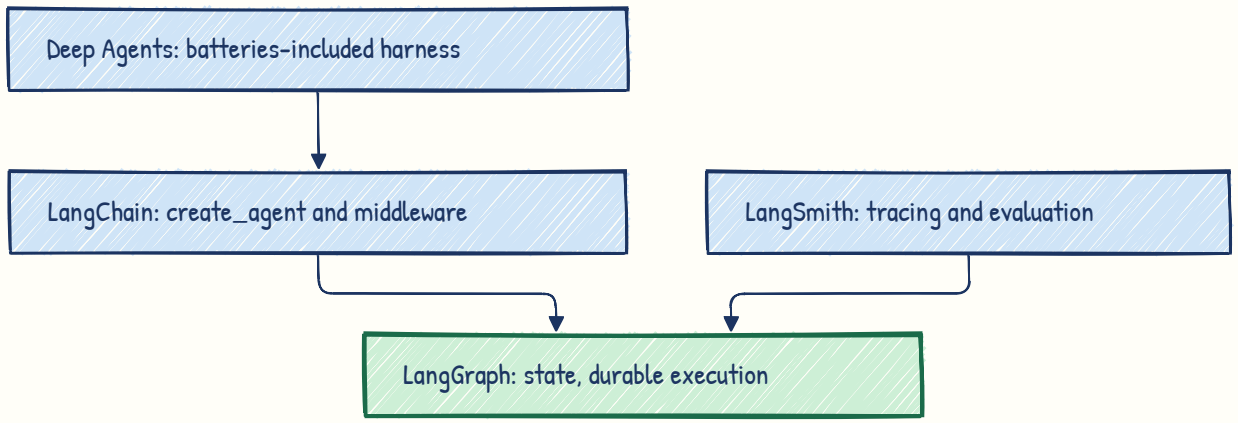


Figure 1. The LangChain family as its docs describe it. Higher layers build on LangGraph, and LangSmith observes any of them.

Layer	What it is, per the docs	Choose it when
LangGraph	"a low-level orchestration framework and runtime for long-running, stateful agents"	You need direct control of state and flow
LangChain	A framework built on the idea "Agent = Model + Harness". `create_agent` is "a minimal, highly configurable harness"	You want an agent loop with middleware
Deep Agents	A harness with context compression, a virtual filesystem and subagents	You want more built in
LangSmith	Tracing and evaluation, across frameworks	You need observability

"LangGraph does not require LangChain", while "LangChain's agents are built on top of LangGraph."

Older material describing chains and `LLMChain` reflects an earlier LangChain. Check the date on anything you read, including the `LangChain Handbook.pdf` in this folder, which this course treats only as a checklist.

LangChain's agent

Condensed from the docs, and not run here, because it needs a provider key:

```
from langchain.agents import create_agent

agent = create_agent(
    model="claude-sonnet-4-6",
    tools=[get_weather],
    system_prompt="You are a helpful assistant",
)
result = agent.invoke({"messages": [{"role": "user", "content": "Weather in Pune?"}]})
```

This is the loop from AI06 L2, in one call. **Middleware** is the place for guardrails, retries, routing and custom tool policies, which fills a similar role to hooks (AI06 L8). Confirm that it can provide the guarantee you need, and enforce money rules in the backend as well.

LangGraph's building blocks

Concept	What it is	Equivalent in AI06
State	A shared typed dictionary	Case state
Node	A function that takes state and returns updates	A step
Edge	A fixed link, or a conditional one	Code-controlled path
Checkpointer	Saves state per thread	Manifest, saved conversation
Thread	One run's identity, `thread_id`	Case ID
Store	Long-term data across threads	Memory
`interrupt()`	Pause, and return a payload to the caller	Approve before action

Here's the graph from the lab. **Note how deterministic steps and an approval step share one graph**, which is a workflow in AI06 L1's terms:

```
builder.add_conditional_edges("check", route, ["approve", "refund"])
builder.add_edge("approve", "refund")
builder.add_edge("refund", END)
graph = builder.compile(checkpointer=InMemorySaver())
```

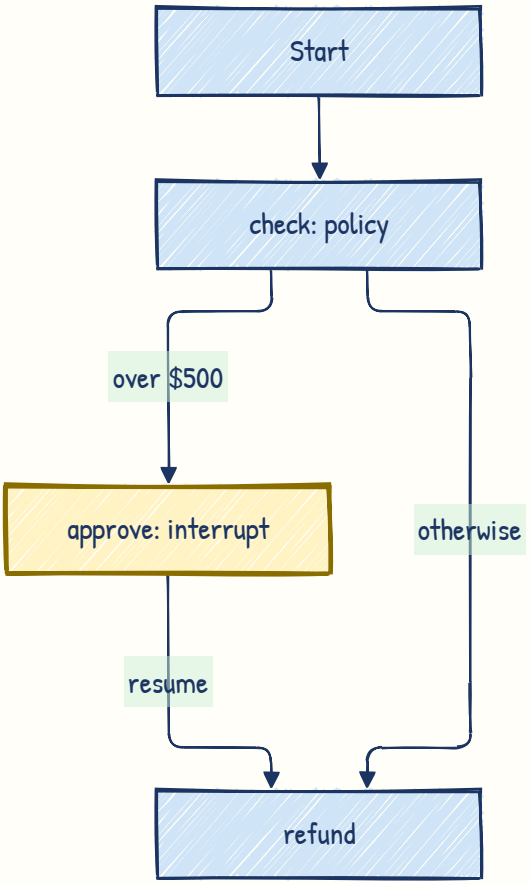


Figure 2. The lab's graph. A large refund pauses at the approve node, with state saved, until a person resumes it.

Checkpointers

Saver	Use	Note
`InMemorySaver`	Tests	Lost on restart
`SqliteSaver`	Local development	A file
`PostgresSaver`, `AsyncPostgresSaver`	Production	`thread_id` under 255 characters

Prune them. The docs warn of unbounded growth, and this short lab already saved 5 checkpoints for one case.

The rules of `interrupt()`

From the interrupts docs:

Don't	Do instead	Why
Send emails or do inserts before `interrupt()`	Move them after it, or into another node	The node restarts from the top
Wrap `interrupt()` in a bare `try/except`	Call it outside the try block	It catches the pause exception
Reorder or conditionally skip `interrupt()` calls	Keep the same order every run	Matching is by index
Use `while True` for validation	One interrupt per node, and a conditional edge	Each resume replays earlier iterations
Pass objects to `interrupt()`	Pass JSON-serializable data	It's stored
Use a new `thread_id` to resume	Reuse the same one	A new ID is an empty thread

Static breakpoints (`interrupt_before`) are for debugging, and the docs don't recommend them for human approval.

Why the obvious fixes fall short

Tempting move	Why it disappoints	Better
Hand-rolled status table for approval flows	It works, and then you rebuild retries, replay and history	Fine for one flow. Use a graph or Temporal for many
A graph for a simple linear chain	More concepts than the problem needs	Plain functions
LangGraph as your whole architecture	Couples the business logic to one runtime	Nodes call your own services
Trusting a checkpoint to prevent double side effects	It restores state, and external effects are yours	Idempotency keys

Durable execution isn't new: workflow engines such as Temporal offer it too, in several languages. For a Java team, that may be a better fit than a Python runtime. That comparison comes from general knowledge, so evaluate it for your case.

How the field solved it

Problem	Approach
Agents die mid-run	Checkpoint state each step
Approval takes hours	Pause, and free the process
Mixed deterministic and model steps	One graph with both
What happened in this run?	Tracing, through LangSmith or similar

Anthropic's posts come at this from a different direction, with explicit state files and a simple loop (AI06 L7). Both make state outlive the process, which is the shared lesson.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: a chain in memory. A restart loses everything, and approval can't wait.

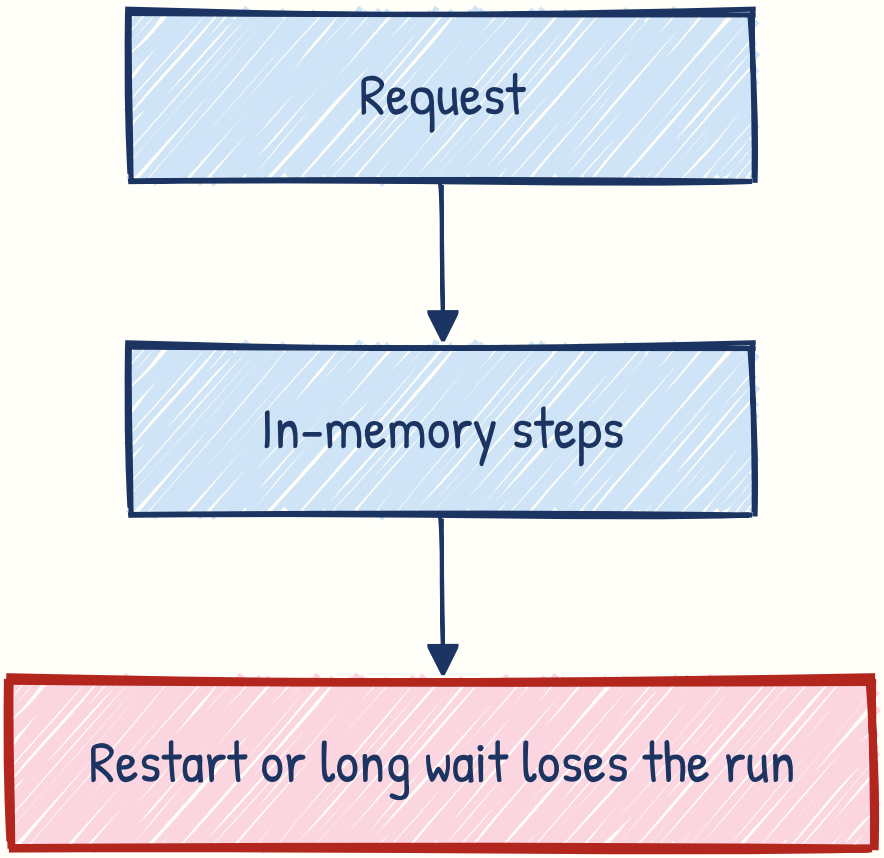


Figure 3. Baseline. A run lives in process memory, so it can't wait hours for a person, or survive a restart.

Improved: a graph with a checkpointer. Runs can pause and resume. New problem: **re-executed nodes can repeat side effects, and graph versions can change while runs are paused.**

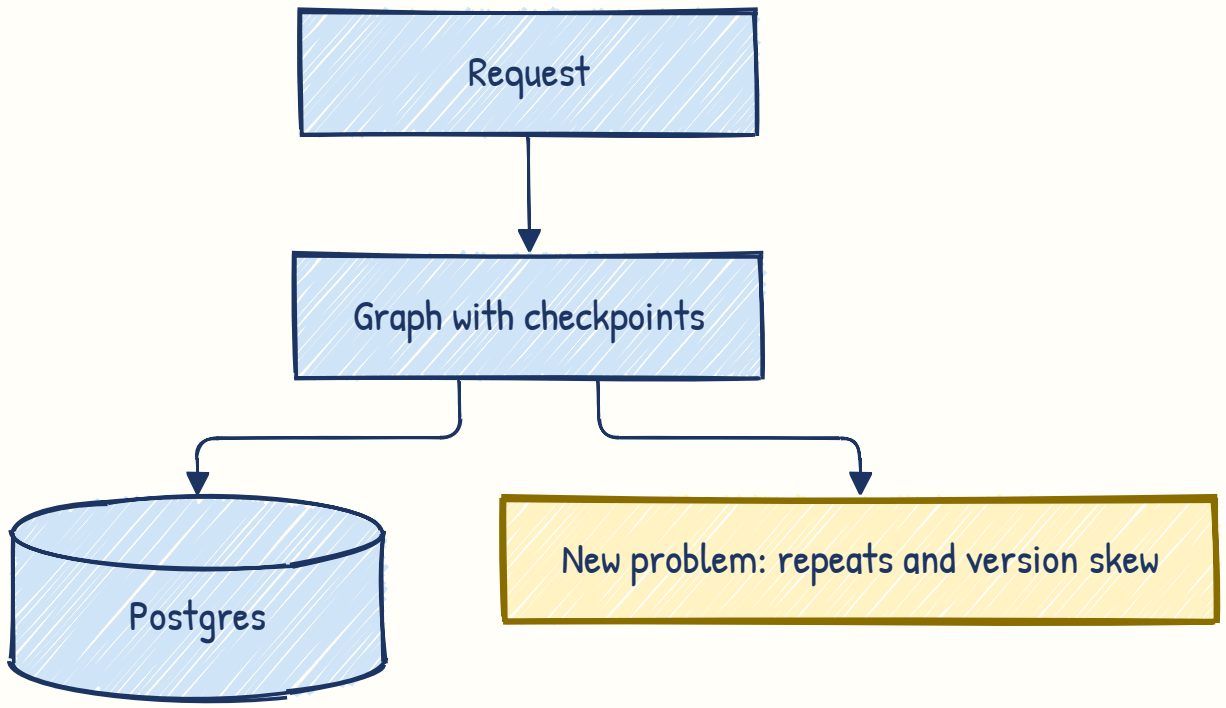


Figure 4. Improved. Saved state survives restarts, but resumed nodes can repeat effects, and old runs meet new code.

Production: thin nodes over idempotent services.

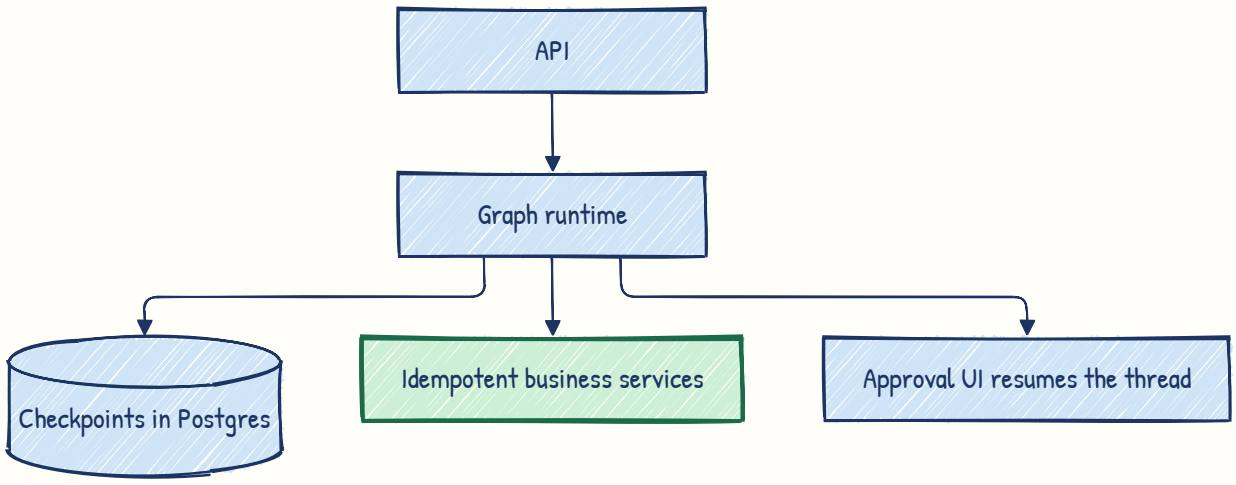


Figure 5. Production. Nodes hold little logic and call idempotent services. Side effects sit after interrupts, and approvals come through your own UI.

Component	Why it exists	How it communicates
Graph runtime	Flow, pause, resume	Library in your service
Postgres checkpointer	Durability	SQL, with retention
Idempotent services	Replays are safe	Called with a key from the state
Approval UI	People resume runs	Uses the `thread_id`
Traces	Debugging	LangSmith or OpenTelemetry

Failures

Failure	Containment
Node repeats an effect	Keep pre-interrupt code idempotent
Paused run meets a changed graph	Version graphs, and drain or migrate old runs
Checkpoints grow forever	Retention policy
Lost `thread_id`	Store it with your business record
Approval never arrives	A timeout path

Java

LangGraph is Python and JavaScript. **There's no Java here, because that would misrepresent the library.** A community port, LangGraph4j, exists, which I haven't evaluated. Java teams have other routes:

Need	Java option
Agent workflows	`langchain4j-agentic`, marked experimental (AI07 L4)
Durable pause and resume	A workflow engine, or your own state table (AI06 L7)
Calling a Python graph	Run it as a service, and call it over HTTP

Trade-offs

Decision	Gain	Cost
LangGraph	Durable, explicit flow	Execution rules to learn, Python or JS
LangChain `create_agent`	Fast, middleware	One more layer
Graph vs plain code	Pause and replay	Harder to read for simple flows
LangSmith	Good visibility	Another vendor. OpenTelemetry may do

When not to use it: a stateless call, a short linear chain, or a Java team with no Python to run.

Common mistakes

1. **Side effects before `interrupt()`.** They repeat.
2. **`InMemorySaver` in production.** It's lost on restart.
3. **A new `thread_id` on resume.** That's an empty thread.
4. **Bare `try/except` around `interrupt()`.** It swallows the pause.
5. **Learning from old tutorials.** The APIs have changed a lot.
6. **Business logic inside nodes.** Keep nodes thin.
7. **Assuming checkpoints make effects safe.** They restore state only.

What to say in an interview

"LangGraph gives me checkpointed state, and a pause that frees the process, which suits approval flows. The catch is that a resumed node runs again from its first line, so I'd keep nodes thin, and put side effects after the interrupt, with idempotency keys. For a linear chain I'd use plain functions, and a Java team might prefer a workflow engine."

Certification lens

How this lesson maps to CCAR-F

Not tested. The ideas carry over: durable state (TS 5.4), and human approval and gates (TS 1.4).

These are original practice questions for understanding, not real or leaked exam questions.

Q Q1 · Code analysis · Medium. A node runs `send_email(state); ok = interrupt("Approve?"); charge(state)`. What happens on approval, and what's the fix?

A The node restarts from its first line, so the email is sent twice. `charge` runs once. Move the email to an earlier node, which won't re-run, or make it idempotent. Also protect `charge` with an idempotency key.

Q Q2 · Best decision · Medium. Choose: (a) one model call, then one database write; (b) a loan process with three human approvals over several days; (c) an open-ended research assistant with no approvals.

A (a) Plain code. (b) A durable graph or workflow engine, since it must survive days and restarts. (c) An agent loop, possibly `create_agent` or the Agent SDK. A graph adds little unless you need explicit control.

Q Q3 · Troubleshooting · Medium. Resuming returns immediately, with no result and no error. `graph.get_state(config).next` is empty. What's the likely cause?

A A `thread_id` that doesn't match the paused run, which gives an empty thread, as the lab shows. Or an in-memory saver that lost state on restart. Store the ID with your business record, and use a durable checkpointer.

Revision - Lesson 2

→ LangGraph: state, nodes, edges, checkpointers. Low-level and durable.

→ LangChain: create_agent and middleware, built on LangGraph.

→ Deep Agents adds more harness. LangSmith traces and evaluates.

→ LangGraph doesn't require LangChain.

→ interrupt() pauses and returns a payload. Command(resume=...) continues.

→ A resumed node restarts from its first line. Finished nodes don't re-run.

→ Side effects go after the interrupt, or in another node.

→ Need a checkpointer and the same thread_id.

→ InMemorySaver is for tests. Use Postgres in production, with retention.

→ Checkpoints restore state, and external effects need idempotency.

→ Old tutorials are often out of date.

→ Java teams might prefer a workflow engine or a separate Python service.

Your notes